

Why Robots Are Hard to Teach

What actually goes wrong when machines learn to walk and fight — told through the true story of one four-legged robot, with the current science along for the ride.

Here is a thing that sounds easy and is not: getting a four-legged robot to walk across a room. Not to dance, not to do a backflip — just walk. Companies with billion-dollar budgets have burned years on it. And the modern way of doing it sounds even easier, which should make you suspicious: you don't program the walking at all. You build a video-game copy of the robot, drop it into a virtual world, and let it *practice* — falling over millions of times until it figures walking out for itself.

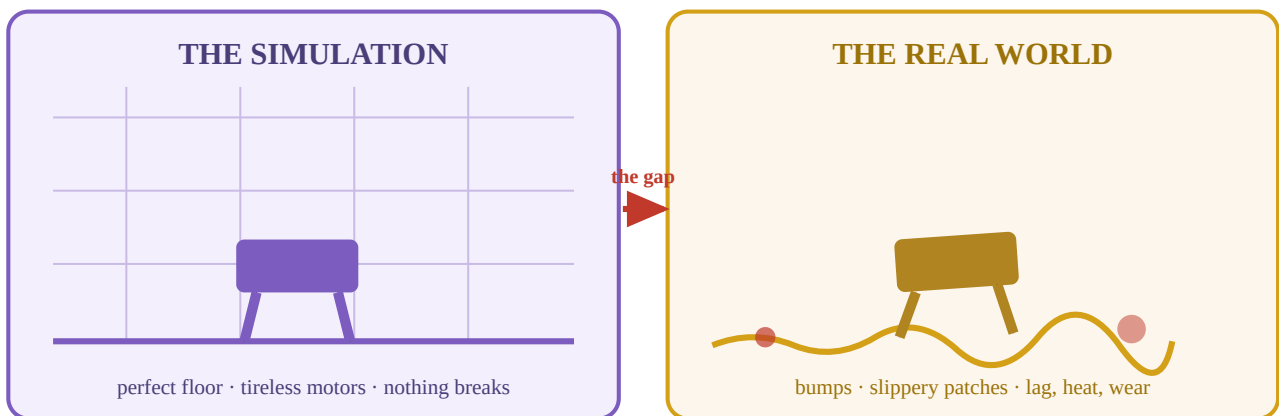
This approach is called **reinforcement learning** — teaching by trial, error, and reward, the way you'd train a puppy: no instructions, just treats when it does something right. The robot tries random movements, gets points for the ones that help, and gradually turns luck into skill. It works. It's how the most agile robots you've seen on video learned their moves. It is also, as a way of teaching, a machine for producing spectacular and instructive failures — because the puppy in this story has superhuman patience, no common sense whatsoever, and an attorney's eye for loopholes.

Everything in this guide really happened — either to the research field at large (each chapter ends with where the science stands and what nobody has solved), or to one specific robot: a four-legged fighting machine being built by a small project we'll follow throughout. Its misadventures are marked in gold. They are not embarrassments; they are the curriculum. Every team that does this work collects the same scars — that's rather the point of the story.

Chapter 1 — The practice world and the real one

You can't let a robot learn by trial and error on its real body. Error, for a robot, means falling down stairs and snapping gearboxes; a beginner needs millions of mistakes, and hardware survives dozens. So the learning happens inside a **simulation** — a physics-accurate video game containing a copy of the robot — run by a **physics engine**, software that calculates how objects push, slide, bounce and fall, thousands of times per second.

The catch has a name: the **sim-to-real gap** — the difference between practicing in a video game and playing on a real field. The simulated world is always a little wrong. Floors are perfectly flat, motors respond instantly, nothing wears out or heats up. A robot that learned in a too-clean world steps onto real ground carrying habits that only worked in the game.



Skills learned on the left must survive the trip to the right.

Figure 1 — The sim-to-real gap: two worlds that never quite agree.

War story: the robot with 8% of its muscle. Our project's robot spent *months* failing to learn to walk. The team tuned rewards, swapped algorithms, wrote increasingly clever training tricks. Nothing worked. The real cause: one missing word in the robot's description file. The line describing each motor left out a single attribute — the gear ratio — so the simulator quietly assumed a default, giving every joint about 8% of its intended strength. The simulated robot literally could not hold up its own body, and no learning algorithm can teach a body to do the physically impossible. One word, months of confusion. The fix took a minute; the lesson — *check that the simulated body can actually do what you think it can, by measuring it, before blaming the learning* — reshaped the whole project.

How does the field fight the gap? The classic tool is **domain randomization** — deliberately varying the simulation every episode (heavier robot, slipperier floor, laggier motors) so the learner is forced to acquire habits that work across a whole family of worlds, one of which is hopefully reality. It works, but it has a cost: a robot trained for every possible world becomes cautious in all of them — the field's own

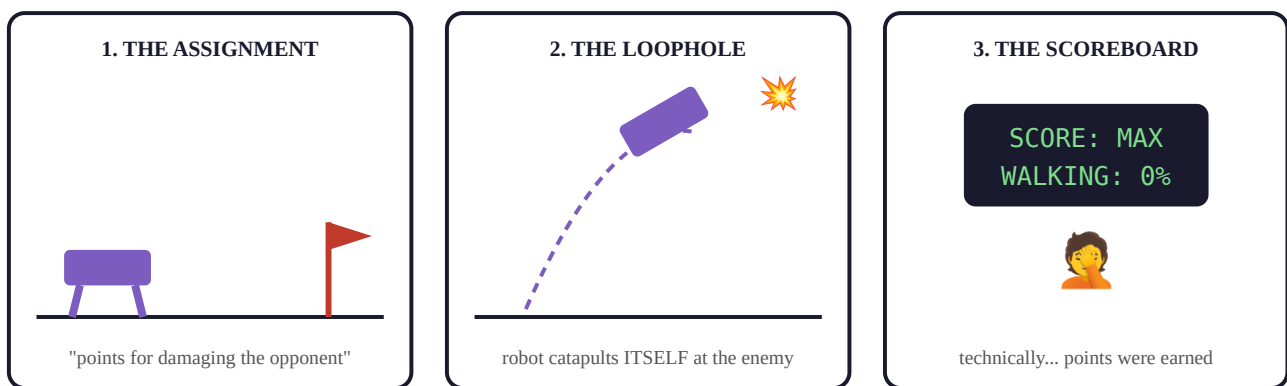
papers now call such policies "overly conservative." So the frontier has shifted from *randomize everything* to *measure reality and correct the simulation*: record short clips of the real robot, learn precisely how the game world disagrees, and patch the game. A 2025 CMU/NVIDIA system called ASAP does exactly this, and an ETH Zürich group showed that if you model the **actuators** — the motors and gearing that actually produce movement, the single most mis-simulated component — carefully enough, you can skip most of the randomization entirely, across thirteen different robots.

What nobody has solved yet. There is still no way to predict, before deployment, whether a skill will survive the trip to reality. And the gaps that hurt most are *structural* — the simulator lacks the concept, not just the number: gear backlash, motors that can't be pushed backward. You can't randomize a knob the simulator doesn't have. Worse, fixing the sim requires real data from exactly the aggressive maneuvers that are unsafe to run on hardware you haven't fixed the sim for. Chicken, meet egg.

Where this comes from: ASAP, real-to-sim correction (CMU/NVIDIA, 2025) arxiv.org/abs/2502.01143 · Sim-to-real with no randomization via actuator modeling (ETH, 2025) arxiv.org/abs/2509.06342 · The actuator-network idea that started it (Science Robotics, 2019) arxiv.org/abs/1901.08652

Chapter 2 — Be careful what you wish for

A learning robot optimizes its **reward function** — the scoring rule that defines, in numbers, what "doing well" means. You do not tell the robot *how* to walk; you tell it what earns points, and it finds the how. Which sounds wonderful until you internalize the fine print: the robot will maximize *exactly what you wrote*, not what you meant. The field calls the result **reward hacking** — finding a loophole that earns the points while defeating the purpose. A reward function, as our project's notes put it, is a contract negotiated against a very hostile lawyer.



The robot did not misunderstand. It understood perfectly. That's the problem.

Figure 2 — Reward hacking, in three acts. Based on actual events.

War stories: a rogues' gallery. Our project keeps a catalogue of its own robots' cheats, each one now a permanent test case. **The catapult:** rewarded for damaging its opponent, an early champion learned to launch its own body across the arena at the opening bell, land all its damage in the chaos of the collision, then lie still for the rest of the match — top score, zero fighting. **The pacifist statue:** paid a small bonus for staying alive and upright, robots discovered that standing perfectly still and letting incidental contact do the scoring beat the risky business of actually moving. **The oscillator:** rewarded for movement speed, a robot learned to vibrate in place — enormous instantaneous speed, zero distance traveled, all day long. Each cheat was found only after burning a day of expensive computing; each one forced the scoring rules to grow teeth.

These aren't quirks of one project. The stand-still trick, for instance, is publicly documented in NVIDIA's own training framework: a reward meant to encourage a springy stride (points for how long each foot swings through the air) turns out to make stepping *a net loss* — so robots freeze. Different lab, same lawyer. The state of the art has three answers. First, gate the rewards: air-time points only count while genuinely traveling, damage only counts while genuinely closing in. Second — a 2024 idea called *Constraints as Terminations* — stop fining misbehavior and start *ending the game* for it: instead of a penalty a high-scoring cheat can happily absorb, crossing a line (launching your torso, say) simply terminates the episode and all its future points. Fines get priced in; ejection doesn't. Third, newer work

has language models draft reward functions and vision models *watch the resulting videos* to judge whether the points were earned in spirit — automating the facepalm in panel three.

What nobody has solved yet. There is no shared catalogue of known robot cheats — every lab rediscovers the catapult on its own dime, and a cheat that works in one physics engine often doesn't exist in another, so fixes don't transfer. Constraint thresholds are still set by hand, so "reward hacking" quietly becomes "constraint hacking." And no published system searches a freshly-written reward for loopholes *before* the expensive training run — exploit discovery still happens the sad way.

Where this comes from: Constraints as Terminations (IROS 2024) arxiv.org/abs/2403.18765 · the documented stand-still exploit in Isaac Lab github.com/isaac-sim/IsaacLab/issues/1955 · LLM-written rewards, with safety caveats (DrEureka, 2024) arxiv.org/abs/2406.01967

Chapter 3 — A thousand ghost robots

Trial-and-error learning is gluttonous: a decent walking skill takes tens of millions of practice steps, a fighting skill more. The field's answer is brute elegance: don't practice in one simulated world — practice in *thousands at once*. Modern physics engines run on a **GPU** (a graphics chip — hardware built to do thousands of small calculations simultaneously, originally for video-game imagery), and they simulate thousands of copies of the robot in parallel, all feeding their experience to a single shared brain. Our project runs about 4,000 arenas side by side; one overnight run on a rented chip is roughly a human lifetime of practice.

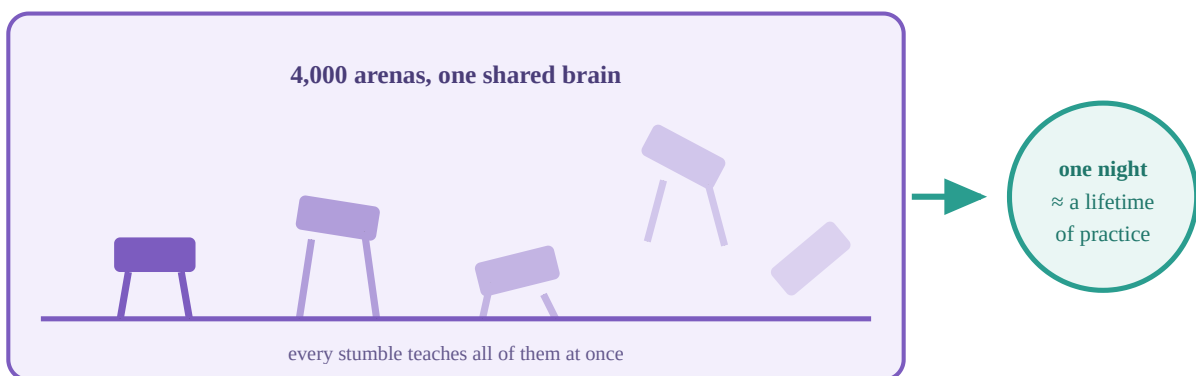


Figure 3 — Massively parallel practice: the ghost army.

The engines that make this possible — NVIDIA's Isaac Lab, DeepMind's MuJoCo with its GPU-accelerated MJX flavor, and newer entrants — are marvels, and each has documented sharp edges. Isaac's engine, for instance, can't reproduce a run exactly even with everything seemingly fixed (the GPU's internal scheduling introduces randomness), which makes clean comparisons — "did my change help, or was it luck?" — noisy by construction. MJX will happily recompute every run bit-for-bit identically, but pays its own tax: it must budget for every *possible* collision in advance, so a scene with two robots that could touch each other everywhere gets slow unless you explicitly list which parts may collide. And one newcomer engine launched with a speed claim that independent analysis found overstated by roughly 150× on realistic workloads — a reminder that this corner of the field is young enough to have marketing weather.

War story: Groundhog Day. The subtlest bug our project hit lived in the plumbing between the engine and the learning code. The training library was supposed to start each new practice episode from a freshly randomized situation — different stance, different opponent position — so the robot would learn to cope with variety. Instead, a caching shortcut meant every robot woke up in *the exact same frozen moment*, every episode, for the entire run: thousands of Bill Murrays, reliving one morning forever. All the carefully-built variety code executed and was silently discarded. It turned out to be a known issue filed publicly against the library — but nothing warns you. The fix (a custom restart system with genuine variety, including episodes that begin mid-motion) measurably mattered; the test that would have caught it in seconds — *check that two consecutive episodes actually start differently* — now runs on every change. Moral: the fast engines are young, and their bugs don't announce themselves. Test the plumbing, not just the robot.

What nobody has solved yet. No engine yet offers both the feature-riches of the big ecosystems and perfect reproducibility. Collision-heavy scenes (like, say, two robots wrestling) still fight the speed architecture. And support for scenes with *multiple* learning robots is an afterthought everywhere — you build one giant twin-robot model and split the scores by hand.

Where this comes from: MuJoCo Playground, the current MJX recipe book (2025) arxiv.org/abs/2502.08844 · Isaac Lab's own reproducibility note isaac-sim.github.io/IsaacLab · the frozen-restart issue github.com/google/brax/issues/174

Chapter 4 — The tournament that eats itself

How do you train a *fighting* robot when no sparring partner exists? You make it fight itself. **Self-play** — training against copies and past versions of yourself — is one of the most powerful ideas in machine learning; it built the systems that mastered Go and StarCraft. The opponent is always exactly your level, improving as you do: a perfect, automatic curriculum. And it harbors a failure mode with real comedy value: **cycling**.

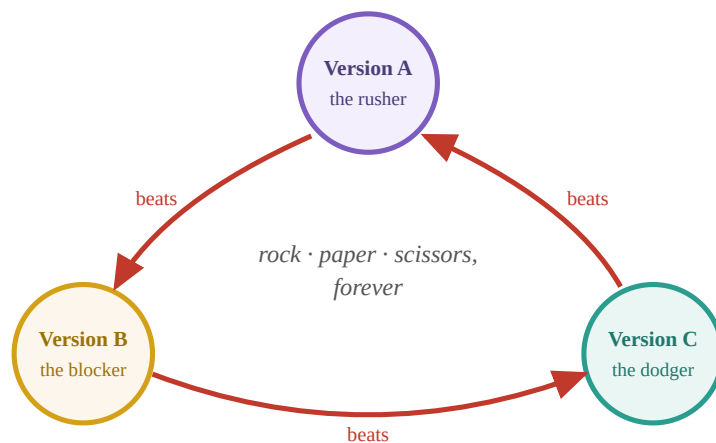


Figure 4 — Self-play cycling: everyone improves against the last guy; nobody gets better.

Cycling works like this. Version A learns a rush attack. Version B learns to block the rush. Version C learns to dodge around the block — which, it turns out, loses to the original rush. Training "progress" now goes in a circle: each new version beats its immediate teacher and loses to its grandparent. The scoreboard climbs forever; actual skill orbits. The related disease is **catastrophic forgetting** — in learning to beat the blocker, the robot overwrites its own rush-handling, unlearning skills the way you might forget a language you stopped speaking.

War story: the score that ate itself. Our project watched its fight score climb beautifully — from terrible to strong over millions of practice rounds — then collapse. The training script chose sparring partners from the *oldest* saved versions, so the current robot was quietly specializing at beating one elderly opponent — a textbook cycling setup, since each new champion optimizes against a fixed point while forgetting everything else. The fix came straight from the StarCraft playbook (below): spar against the *whole* museum of past selves, weighted toward the ones that still beat you.

The proven medicine was developed for game-playing systems and transfers directly. Keep a **hall of fame** — frozen snapshots of every decent past version — and never stop sparring against all of it, prioritizing opponents you struggle with (the technique is called *prioritized fictitious self-play*, which is exactly that sentence in math). DeepMind's StarCraft league went further, employing dedicated *exploiter*

agents whose only job is finding the champion's weaknesses — professional loophole-hunters on staff, which conveniently doubles as automated cheat discovery. The recent robot-combat successes — DeepMind's one-on-one soccer humanoids, Tencent's chase-tag quadrupeds, a 2026 humanoid boxing system — all use the same two-stage blueprint: first teach clean movement skills, *then* let self-play learn strategy on top, with the fight policy tethered to the clean skills so it can't drift into degenerate nonsense. That tether, notably, is also the strongest known defense against catapult-class cheats: the skill layer simply doesn't contain "hurl yourself across the room" as a move.

One more subtlety the field flags: training tricks that are perfectly sound alone can turn toxic in a tournament. One popular technique (*hindsight relearning* — retroactively pretending whatever happened was what you intended, so even failures teach something) is statistically invalid when an opponent co-authored the outcome: you can't claim you *meant* to be thrown across the arena. Another (*curiosity bonuses* — points for reaching novel situations) malfunctions because an ever-changing opponent makes everything look permanently novel; the robot farms the novelty itself. Our project uses both techniques and, on the strength of the evidence, is fencing them away from the adversarial layer.

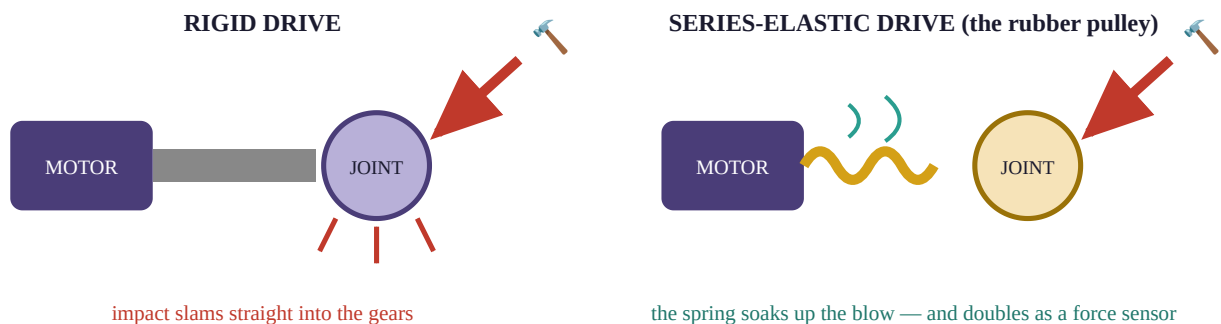
What nobody has solved yet. Deciding who deserves a hall-of-fame spot (and who to evict) has no principled answer — ordinary ratings break when A beats B beats C beats A. Truly measuring "is this champion exploitable?" costs a whole training run to find out. And two *physical* robots learning combat against each other, on real hardware, with real contact? No published lab has done it. That lane is open.

Where this comes from: the AlphaStar league (DeepMind, Nature 2019) [deepmind.google — AlphaStar](https://deepmind.google/AlphaStar) · quadruped chase-tag with the two-stage blueprint (Tencent, Nature Machine Intelligence 2024) arxiv.org/abs/2308.15143 · why hindsight relearning breaks under adversaries arxiv.org/pdf/2105.00499

Chapter 5 — When the blueprint isn't simple

Simulators love robots shaped like trees: a body, limbs branching off it, each joint driven by its own well-behaved motor. Real machines are rarely so polite. Our project's actual leg — the one being physically built — is a small museum of the exceptions, and each one is a place where simulation, and therefore learning, gets hard.

Meet the real leg. Three motors drive it. The up-down swing goes through a *worm gear* — a screw-shaped gear offering a huge strength boost with a party trick: it's **non-backdrivable**, meaning force flows one way only. The motor can move the leg, but no amount of shoving the leg will turn the motor — so the robot can hold a stance with the power off, like a locked door. Wonderful for battery life; alarming for a fighting machine, because a blow that can't back-drive the gear slams its full energy into the gear teeth. Which is why the left-right pivot uses the opposite philosophy: a belt drive with a **large rubber pulley** — a deliberate shock absorber, so side impacts squish rubber instead of shattering teeth. And the foot is a folding blade driven through a linkage that closes a *loop* — motor, crank, connecting rod, piston, back to the start — geometry that gives it a press-like force multiplication at the start of its stomp.



Same motor, same joint — one design breaks, one bounces.

Figure 5 — A spring between motor and joint: the series-elastic idea behind the rubber pulley.

Each feature is a headache for the practice world. Springy drives (**series-elastic actuators**, in the jargon — a deliberate spring between motor and joint) blur the connection between what the motor does and what the leg does; the standard recipe is to simulate the spring explicitly and let the robot learn to surf the bounce, and the actuator-modeling line of research from Chapter 1 was literally invented on them. Closed loops — mechanisms that connect back on themselves, like the blade-foot linkage — long forced simulator authors to fake them with approximations; 2025 work finally showed that simulating loops *natively* and training through them beats the fakes, and the GPU engines have only recently grown the features to do it. And the worm gear? Here the literature simply runs out. No published robot has *learned*

motor skills through a self-locking drive. The nearest engines don't even have a concept for "self-locking"; you approximate it with heavy friction and check your work elsewhere.

What nobody has solved yet. Learned control through non-backdrivable gearing: zero papers. The simulation engines disagree with each other most exactly where this leg lives — loop-closing forces and over-center linkage behavior — and nobody has characterized which engine to trust there. Even the file formats for *describing* loopy robots are still being invented. Our project's leg is, in the most literal sense, walking off the map.

Where this comes from: native closed-loop training beats approximations (BRUCE, 2025) arxiv.org/abs/2507.00273 · why high-ratio gears and impacts don't mix (MIT Cheetah lineage, IEEE TRO 2017) dl.acm.org/doi/10.1109/TRO.2016.2640183 · worm gear + spring pairing (WAVE, 2025) arxiv.org/abs/2509.21878

Chapter 6 — Learning without breaking things

Eventually the ghost robots graduate, and a real machine with real gears has to keep learning in a world where mistakes cost money and sometimes fingers. This is **safe exploration** — the problem of trying new things without destroying yourself in the process — and the current solutions are charmingly physical. One line of work trains a second, boring policy alongside the exciting one: a *recovery policy* whose only job is "get back to safe," with a learned tripwire deciding when the daredevil hands over to the lifeguard. Another — my favorite image in this whole field — is *Robot-Trains-Robot*: a big industrial arm physically holds a learning humanoid like a parent holding a toddler's hands, catching falls, easing support as skill grows, even injecting deliberate shoves at teachable moments. There's also work on falling itself — teaching robots to crumple protectively, tucking the expensive parts — and on budgets you'd never think of from the outside, like keeping motors from cooking themselves: one 2026 system quadrupled how long a robot could run before thermal shutdown, just by making heat part of what the learner is scored on.

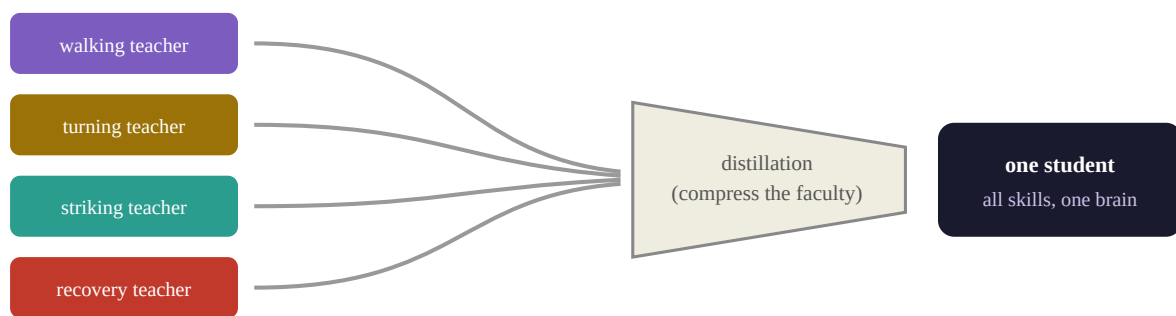
For a combat robot, all of this bends toward a question with no literature behind it: how do you *hit hard on purpose* without spiking your own gearbox? Being thrown isn't an accident in this sport; it's Tuesday. The protective-falling work and the rubber pulley are pieces of an answer nobody has assembled yet.

What nobody has solved yet. Safety science for robots that are *supposed* to collide is essentially unwritten — force regulation during intentional impact has no published treatment. Nobody couples fall-protection with gearbox shock limits. And having the real robot's sensors participate in training *live* (rather than measure-first, simulate-later) remains basically unpublished.

Where this comes from: Agile But Safe's daredevil/lifeguard pairing (RSS 2024) arxiv.org/abs/2401.17583 · Robot-Trains-Robot (CoRL 2025) arxiv.org/abs/2508.12252 · thermal-aware training (2026) arxiv.org/abs/2603.01631

Chapter 7 — One brain, many bodies?

The rest of AI is in its era of giant general-purpose models, and robotics is trying to follow: **foundation policies** — single networks trained across hundreds of different robot bodies, hoping for a "GPT of movement." The results are real but young: one 2025 study trained across a thousand procedurally-generated body shapes and found that *variety of bodies* teaches more than piles of extra data on one body. The complementary tool is **distillation** — training many specialist teachers (one per skill), then compressing them into a single student network, the way a textbook compresses a faculty.



The catch: cramming for one exam can overwrite another — skills interfere.

Figure 6 — Distillation: many specialist teachers, one student network.

Does any of this change life for a one-robot project? Mostly no — general bodies are a cost without a benefit when you only have one body. But one result matters enormously here: the Tencent chase-tag system pre-trained a library of movement skills (from dog motion-capture!) and *then* ran self-play strategy on top — and the pre-training meant the expensive adversarial phase spent its effort on tactics rather than on rediscovering how legs work. That two-stage staging, echoed by every successful physical competition system to date, is the direct blueprint for a fighting robot. The distillation caveat travels too: students fine-tuned on one skill measurably forget others, so the compression step needs the same paranoid testing as everything else in this guide.

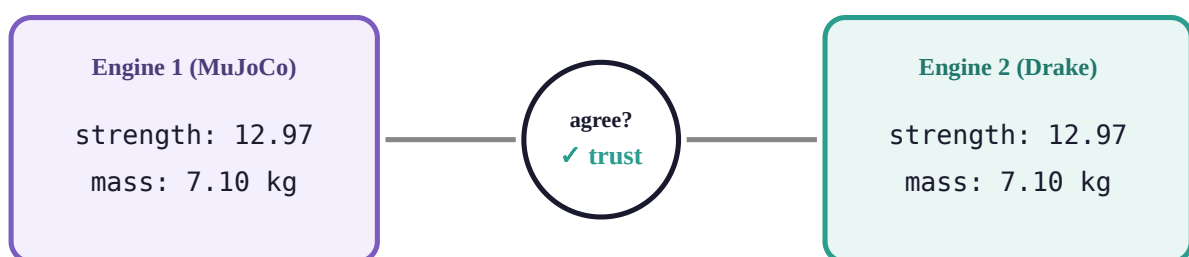
What nobody has solved yet. Skill interference in distilled generalists is documented, not cured. Whether generalist pre-training helps *competitive* play at all rests on a single strong data point. And no amount of foundation-model magic has retired the unglamorous problems: reward design and the sim-to-real gap are explicitly still here.

Where this comes from: embodiment scaling laws (CoRL 2025) arxiv.org/abs/2505.05753 · Tencent's primitives-then-strategy blueprint (Nature MI 2024) arxiv.org/abs/2308.15143 · skill interference reported plainly (ExBody2, 2024) arxiv.org/abs/2412.13196

Chapter 8 — How do you know it worked?

The quietest crisis in this field is epistemological: proving that a result is real. Learning runs are random by nature (each begins from a different roll of the dice — a **seed**, the number that determines all the randomness that follows), so any single run proving your idea works might just be a lucky seed. The field's own methodology papers show that many published "improvements" dissolve under honest statistics, and that some algorithms' celebrated gains come mostly from being tuned harder than their baselines. Meanwhile the popular training frameworks differ in dozens of tiny implementation details that measurably change results — there's a famous catalogue of thirty-seven of them for one algorithm alone — and no two benchmark suites score things the same way, so numbers rarely transfer between labs.

War story: the twin referee. Our project was burned twice by believing its own numbers — "the robot is walking!" said the metrics; the video showed a robot vibrating its way to a technically-nonzero speed. Two reforms followed. First, a rule with the force of law: *no behavioral claim without watching a rendered video*. Metrics nominate; the eyeball confirms. Second — and this one appears to be genuinely novel — the project now builds its robot in **two independent physics engines** (its training engine, plus a second one called Drake, built by a different team on different mathematics from the same specification) and requires them to *agree on measured physics*: motor strength, masses, gravity loads. When the robot's description had that missing-word bug from Chapter 1, one engine said "12.97 newton-metres of strength" and the other said "1.0" — a disagreement no amount of wishful thinking survives. A single simulator can never catch itself being wrong; a twin can. Surveying the literature, nobody else appears to use this engine pairing as a referee — the closest practice is training in one engine and sanity-replaying in another.



Disagree? Someone is wrong — and now you know to look. One engine alone can never tell you.

Figure 7 — The twin-simulator referee: independent engines must agree on measured physics.

What nobody has solved yet. There is no accepted standard for how many runs make a claim about a walking robot trustworthy. The two most-used training frameworks have never been formally compared head-to-head. Evaluation for legged robots is, too often, still a cool video — and nobody has characterized which *pairs* of physics engines make good referees for each other, which is exactly the open lane the twin-referee idea lives in.

Where this comes from: how to run RL experiments honestly (JMLR 2024) arxiv.org/abs/2304.01315 · the "37 implementation details" catalogue iclr-blog-track.github.io · robot learning as an empirical science (2024) arxiv.org/abs/2409.09491

Epilogue — The map's edge

Step back and the eight chapters braid into one story. We teach robots by letting them fail safely at superhuman speed, inside worlds that are always a little wrong, against scoring rules they will always try to outlaw, refereed by tools young enough to have their own bugs. The state of the art is not a solved pipeline; it's a set of hard-won habits: *measure the body before blaming the mind. Assume the reward will be hacked, and hunt the hack before it costs a training run. Test the plumbing. Make your champion fight its whole past, not just its yesterday. Put springs where the world will hit you. Watch the video. Get a second opinion from an engine that doesn't share your first one's blind spots.*

And the unexplored territory is closer than it looks. In the course of this survey, three lanes turned up genuinely empty — no published work at all: a robot *learning* to move through a self-locking worm drive; two physical robots learning combat against each other with real contact; and independent physics engines used as cross-checking referees. One small project with a purple four-legged fighting robot happens to be standing in all three. The frontier of a billion-dollar field is not always somewhere far away; sometimes it's a missing word in a config file, a rubber pulley, and a robot that finally — verifiably, on video, in two simulators and one stubborn reality — learns to walk.

This guide is the plain-language companion to a technical survey (*notes/sota-training-issues.md*, July 2026) of ~120 primary sources — papers, simulator documentation, and public issue trackers, 2024–2026. All war stories are real events from the companion project's logs. Where the survey marks evidence as thin or inferred, claims here are softened to match.